How Computers Work · Section 4

The CPU, Memory, and Storage

By TOUCH DEN · November 11, 2026

Contents

What You Will Learn 1

Before You Start 1

1. The CPU and the Fetch–Execute Cycle 1

2. Registers 2

3. RAM: Fast and Temporary 2

4. Storage: Slower and Permanent 3

5. The Memory Hierarchy and Cache 4

6. Clock Speed, Cores, and Threads 4

7. Why "Volatile" Matters (a Forensics Preview) 5

Common Mistakes 5

Summary 6

Check Your Understanding 6

Practice 7

What You Will Learn

- What the CPU is, and the fetch–execute cycle it repeats.
- What registers are.
- RAM: fast, working memory that forgets when power is lost.
- Storage: slower memory that keeps data without power.
- The memory hierarchy, and why cache exists.
- Clock speed, cores, and threads.
- Why "volatile" memory matters for forensics.

Before You Start

Read lessons 1 to 3. You know everything is bytes. Now we look at the hardware that *works on* those bytes: where they are processed (the CPU) and where they are kept (memory and storage). No code in this lesson; it is about understanding the machine.

---

1. The CPU and the Fetch–Execute Cycle

The **CPU** (Central Processing Unit), or **processor**, is the part of the computer that actually follows instructions. It is the "brain", though as lesson 1 said, it does not think; it obeys. Everything a computer does, the CPU does, by carrying out instructions one after another.

The CPU works by repeating a simple loop, billions of times a second, called the **fetch–execute cycle**:

1. **Fetch:** get the next instruction from memory.
2. **Decode:** work out what that instruction means.
3. **Execute:** do it (add two numbers, move some data, compare two values).
4. Repeat, with the next instruction.

That is the heartbeat of every computer. A program is just a long list of these tiny instructions sitting in memory, and the CPU marches through them, fetch, decode, execute, fetch, decode, execute, faster than you can imagine. One instruction does almost nothing on its own ("add these two numbers"); billions of them per second add up to a running game or a video call.

This loop is the foundation of reverse engineering and exploitation. A reverse engineer reads the very instructions the CPU fetches. An exploit works by changing *which* instruction the CPU fetches next, redirecting that relentless loop to the attacker's own instructions. Hold onto the image of the CPU as a tireless worker reading instructions from memory in order; later courses are about controlling what it reads.

2. Registers

The CPU needs somewhere to hold the few values it is working on *right now*, the two numbers it is about to add, the result, a note of which instruction comes next. For that it has **registers**: a very small number of tiny, extremely fast storage slots built directly into the processor.

Registers are the fastest memory in the computer, because they are inside the CPU itself. But there are very few of them (a few dozen), and each holds only a few bytes. They are a workbench, not a warehouse: just enough room for the job in hand. When the CPU needs a value that is not in a register, it fetches it from the larger, slower memory outside.

A couple of registers are special and worth knowing by name, because they star in later courses:

- The **program counter** (or instruction pointer) holds the address of the *next instruction* to fetch. Change it, and you change what the CPU does next, which is the goal of many exploits.
- The **stack pointer** tracks the top of the stack, a region of memory we meet in lesson 6 and bend in Binary Exploitation.

You do not need the details now. Just know registers exist: tiny, ultra-fast scratch slots inside the CPU, and that one of them decides which instruction runs next.

3. RAM: Fast and Temporary

Registers are tiny, so the CPU needs a much larger place to hold the program it is running and the data it is working on. That place is **RAM** (Random Access Memory), usually just called **memory**.

RAM is where a running program lives: its instructions and its data are loaded into RAM, and the CPU reads and writes RAM constantly as it runs. "Random access" means the CPU can jump straight to any location, by its **address** (a number naming that spot in memory), without reading through everything before it.

RAM has two defining qualities:

- **It is fast** (far faster than a disk), so the CPU is not kept waiting.
- **It is volatile:** when the power goes off, RAM is wiped clean. Everything in it is lost.

That volatility is why you lose unsaved work when the power cuts out: the document was in RAM, not yet written to permanent storage. Volatility is also hugely important in security, which topic 7 returns to: the live secrets of a running system, passwords, keys, decrypted data, are in RAM, and vanish when the machine powers off. This is why investigators capture a suspect's RAM *before* pulling the plug.

4. Storage: Slower and Permanent

RAM forgets when the power goes off, so a computer needs somewhere to keep things permanently: your files, your photos, the operating system itself, everything that must survive a restart. That is **storage** (also called a **drive** or **disk**): the hard drives (HDDs) and solid-state drives (SSDs) in computers, and the flash storage in phones.

Storage is the opposite of RAM in the two qualities that matter:

- **It is slower** than RAM (though modern SSDs are fast, they are still far slower than RAM, and a spinning hard drive is much slower again).
- **It is non-volatile (persistent):** it keeps its data with the power off. That is the whole point of it.

So the normal flow of using a computer is a constant movement between the two: files live in **storage**; when you open one, it is copied into **RAM** to be worked on; when you save, the changes are written back to **storage**. "Save your work" literally means "copy it from volatile RAM to persistent storage".

The RAM/storage split is central to forensics (course 28). **Disk forensics** recovers data from storage, which persists and can even survive deletion (lesson 7). **Memory forensics** captures RAM, which holds live secrets but disappears on power-off. The two are different disciplines precisely because the two kinds of memory behave so differently.

5. The Memory Hierarchy and Cache

We now have three kinds of memory, and they trade off the same way every time: **faster memory is smaller and more expensive; larger memory is slower and cheaper.** You cannot have it all, so computers use several layers at once, called the **memory hierarchy**:

Level	Speed	Size	Keeps data without power?
Registers (in the CPU)	Fastest	A few dozen slots	No
Cache (in/near the CPU)	Very fast	A few megabytes	No
RAM	Fast	Gigabytes	No
Storage (SSD/HDD)	Slow	Hundreds of GB–TB	**Yes**

The idea is to keep what the CPU needs *right now* in the fast, small levels, and everything else in the slow, big levels. The new entry here is **cache**: a small amount of very fast memory, inside or beside the CPU, that holds copies of the data the CPU has used recently or is likely to use next. Because programs tend to reuse the same data and instructions, cache lets the CPU avoid the slower trip to RAM most of the time, which makes a huge difference to speed.

You do not need to manage any of this, the hardware does it automatically. But the mental model, a pyramid from tiny-and-instant at the top to huge-and-slow at the bottom, explains a lot about how computers perform, and it reappears when you study performance and, in exploitation, cache-timing side channels (advanced topics you will meet later).

6. Clock Speed, Cores, and Threads

How fast is a CPU? Three terms describe it, and marketing often blurs them.

- **Clock speed**, measured in **gigahertz (GHz)**, is how many cycles the CPU ticks per second. A 3 GHz CPU ticks three billion times a second, and it does a little work each tick. Higher is faster, all else being equal, but all else is rarely equal.
- **Cores** are complete processors on one chip. A **quad-core** CPU has four CPUs in one, so it can genuinely do four things at the same time. More cores means more real parallel work, which matters a lot for modern multitasking.
- **Threads** are separate streams of instructions. The operating system schedules threads onto cores. A single core can switch between many threads so fast it *looks* simultaneous, and a core may also run two threads at once (a feature often called hyper-threading). We meet threads properly in lesson 6.

The headline: a modern computer does many things *truly* at once (one per core) and many more *apparently* at once (by switching fast between threads). This is why your computer can run a browser, music, and a download together. It also means, in security, that attacks and defenses can run in parallel, and that reasoning about *order* of events across threads is genuinely hard, which is the root of a bug class called **race conditions** (seen in Web Application Security and exploitation).

7. Why "Volatile" Matters (a Forensics Preview)

We end on the single most security-relevant idea in this lesson, because it decides how a real investigation begins. Recall:

- **RAM is volatile:** it holds the live state of a running system, open programs, decrypted files, typed passwords, encryption keys, network connections, but it is **erased the instant the power is cut.**
- **Storage is persistent:** it survives power-off, but a running system's live secrets may exist *only* in RAM and never touch the disk.

This creates a sharp, practical rule in digital forensics, called the **order of volatility**: when investigating a live machine, capture the most volatile evidence *first*. RAM before disk, because RAM vanishes the moment you pull the plug, while the disk will still be there afterward.

Consider a laptop seized while running, with an encrypted disk. The decryption key is in RAM right now, so the data is readable. Pull the plug, and the key is gone forever; the disk is now an unreadable scramble. A trained investigator knows to **capture the RAM first**, while the machine is still on, precisely because of what this lesson taught about volatile versus persistent memory.

You now understand the hardware that works on bytes: the CPU that executes instructions, the registers and cache that feed it, the RAM that holds the running program, and the storage that keeps everything for next time. Next, we look at the software that manages all of this hardware and lets many programs share it: the operating system.

Common Mistakes

1. Thinking the CPU is "smart". It repeats a simple fetch–decode–execute loop, very fast. Its power is speed and literal obedience, not understanding.

2. Confusing RAM with storage. RAM is fast, working, and volatile (lost on power-off). Storage is slower, permanent. "Saving" copies from RAM to storage.

3. Assuming unsaved work or live secrets are on disk. They may be in RAM only. That is why unsaved work vanishes on a power cut, and why investigators capture RAM first.

4. Reading more cores as simply "faster". More cores means more work *in parallel*; clock speed is work *per tick*. They are different, and both matter.

5. Pulling the plug on a live machine during an investigation. It erases RAM, which may hold the only copy of keys and live evidence. Capture volatile memory first.

Summary

Term	Remember
CPU	Repeats fetch → decode → execute, billions of times a second
Registers	Tiny, ultra-fast slots in the CPU; one picks the next instruction
RAM	Fast working memory; holds the running program; **volatile** (lost on power-off)
Storage	Slower, **persistent** memory; keeps files with the power off
Memory hierarchy	Registers → cache → RAM → storage: faster is smaller; cache speeds up RAM
Clock / cores / threads	Ticks per second / parallel processors / streams of instructions
Order of volatility	Capture the most volatile evidence (RAM) first in a live investigation

Check Your Understanding

1. What four steps does the CPU repeat, and what is that loop called?
2. What are registers, and why are they so fast?
3. Give the two defining qualities of RAM, and the two of storage.
4. What does "save your work" actually do, in terms of RAM and storage?
5. What is cache, and why does it speed up a computer?
6. What is the difference between cores and clock speed?
7. A laptop is seized while running, with an encrypted disk. Why must the investigator capture RAM before turning it off?

Answers

1. Fetch, decode, execute (then repeat). It's called the fetch–execute cycle.
2. Tiny, very fast storage slots built into the CPU itself. They're the fastest memory because they're inside the processor, but there are very few and each holds only a few bytes.

3. RAM: fast and volatile (lost on power-off). Storage: slower and persistent (kept with power off).
4. It copies the data from volatile RAM, where the program was working on it, to persistent storage, so it survives a restart.
5. A small amount of very fast memory in or near the CPU that holds recently or soon-needed data, so the CPU avoids the slower trip to RAM most of the time.
6. Cores are complete processors on one chip (more cores = more true parallel work); clock speed is how many cycles the CPU ticks per second (work per tick). Different things.
7. Because the disk's decryption key is in RAM while the machine runs; RAM is volatile, so turning it off erases the key forever, leaving the disk unreadable. The live evidence must be captured first (order of volatility).

Practice

1. Sort these four into the memory hierarchy, fastest/smallest first: RAM, storage (SSD), registers, cache. For each, say whether it keeps data without power.
2. Explain to a non-technical friend, in two sentences, why they lost their unsaved document when the power cut out, using the words RAM and storage.
3. A phone is advertised as "8-core, 3.2 GHz, 8 GB RAM, 256 GB storage". In one sentence each, say what the 8-core, the 3.2 GHz, the 8 GB, and the 256 GB each tell you.

A Worked Example

1. Registers (no) → cache (no) → RAM (no) → storage/SSD (**yes**). Only storage keeps data without power.
2. "Your document was being worked on in RAM, which is fast but temporary and is wiped the instant the power goes off. It had not yet been saved, which would have copied it to storage, the permanent memory that survives a restart."
3. 8-core: it has eight processors, so it can do eight things truly at once. 3.2 GHz: each core ticks 3.2 billion times a second. 8 GB RAM: it can hold 8 gigabytes of running programs and data in fast, volatile working memory. 256 GB storage: it can keep 256 gigabytes of files permanently.

Course mark icon from Material Symbols by Google, Apache License 2.0.